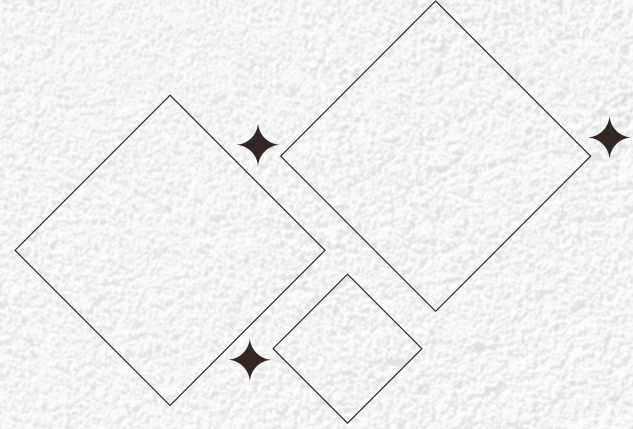


**UNIVERSIDAD
POLITÉCNICA**
DE SAN LUIS POTOSÍ

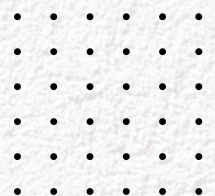
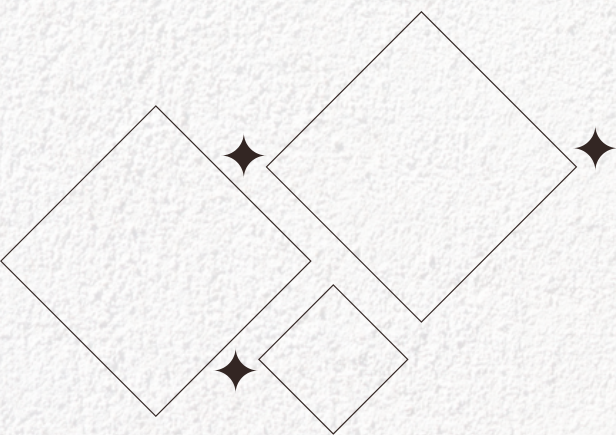


Laboratorio 12- VALIDATION OF FILE EXTENSION WITH NULL BYTE BYPASS

CNO IV Programación WAP

Maestro: Servando López Contreras

Lopez Alcocer Monica Isabel - 179827



Índice

Índice	1
1. Ficha técnica del laboratorio.....	2
2. Objetivo del laboratorio.....	2
3. Marco teórico y fundamentación técnica	2
3.1 ¿Qué es el File Path Traversal?	2
3.2 Validación de la extensión de archivo	2
3.3 El byte nulo (null byte injection): el núcleo del ataque	2
3.4 Variantes de evasión de filtros	3
3.5 Impacto sobre la tríada CIA.....	3
3.6 Clasificación OWASP / CWE	3
4. Explicación técnica de la vulnerabilidad (caso específico).....	3
5. Procedimiento paso a paso.....	4
6. Evidencias — Capturas de pantalla obligatorias	4
6.1 Request interceptado.....	4
6.2 Respuesta vulnerable.....	4
6.3 Confirmación «Lab Solved»	5
7. Explicación del payload.....	5
8. Mitigación recomendada	6
9. Conclusión	6

1. Ficha técnica del laboratorio

Nombre	File path traversal, validation of file extension with null byte bypass
Plataforma	PortSwigger — Web Security Academy
URL	https://portswigger.net/web-security/file-path-traversal/lab-validate-file-extension-null-byte-bypass
Nivel	Practitioner
Tipo de explotación	File Path Traversal — bypass de validación de extensión mediante byte nulo (null byte)
Clasificación	CWE-22 (Path Traversal) · CWE-158 (Null Byte) · OWASP A01
Herramienta	Burp Suite Community Edition (Proxy + Repeater)
Objetivo de resolución	Leer el archivo /etc/passwd del servidor

2. Objetivo del laboratorio

En este laboratorio la aplicación **valida que el nombre de archivo termine con la extensión esperada** .png (por ejemplo product1.png). El objetivo es leer /etc/passwd. La validación es débil porque solo comprueba que la **cadena termine textualmente en** .png, **sin considerar que puede haber un byte nulo** antes. Se explota inyectando %00 para «cortar» la ruta en /etc/passwd mientras la cadena sigue terminando en .png para superar la validación.

3. Marco teórico y fundamentación técnica

3.1 ¿Qué es el File Path Traversal?

El **File Path Traversal** (*Directory Traversal*) permite leer archivos fuera del directorio previsto. En este caso la aplicación no bloquea el recorrido . . /; su única defensa es exigir una extensión de archivo concreta, y esa comprobación se evade con una técnica clásica: la **inyección de byte nulo**.

3.2 Validación de la extensión de archivo

Muchas aplicaciones restringen qué archivos se sirven exigiendo que el nombre **termine con una extensión permitida** (p. ej. «nombre.endsWith(".png")»). La intención es limitar el acceso a imágenes. El problema es que esta comprobación es **puramente textual**: mira los últimos caracteres de la cadena, pero no cómo interpretará esa cadena la capa que finalmente abre el archivo.

3.3 El byte nulo (null byte injection): el núcleo del ataque

En lenguajes de bajo nivel como C, las cadenas terminan en un **byte nulo** (\0, representado en la URL como %00): ese carácter marca «fin de cadena». Muchas plataformas de alto nivel (versiones antiguas de PHP, ciertas APIs) **delegan la apertura del archivo en funciones del sistema escritas en C**. Si el nombre contiene un %00, ocurre un desajuste entre dos capas:

- La **validación de alto nivel** ve la cadena completa `../../../../etc/passwd%00.png` y comprueba que **termina en** `.png` → la acepta.
- La **llamada de bajo nivel al sistema de archivos** interpreta el `%00` como fin de cadena y **trunca la ruta en** `../../../../etc/passwd`, ignorando todo lo que sigue al byte nulo.

Resultado: la validación se supera con la parte `.png`, pero el archivo realmente abierto es `/etc/passwd`. *Históricamente* esta técnica afectó a PHP anterior a 5.3.4 y a diversos componentes en C; el laboratorio reproduce ese comportamiento.

3.4 Variantes de evasión de filtros

Cada laboratorio de la serie ataca una defensa distinta. La de este informe es la primera de la tabla:

Defensa / filtro del servidor	Técnica de evasión
Valida que termine en <code>.png</code> ← este lab	<code>../../../../etc/passwd%00.png</code>
Valida solo el inicio de la ruta	<code>/var/www/images/../../../../etc/passwd</code>
Bloquea <code>../</code> y decodifica URL de más	<code>..%252f..%252f..%252fetc/passwd</code>
Elimina <code>../</code> una sola vez (no recursivo)	<code>....//....//....//etc/passwd</code>
Bloquea <code>../</code> pero permite rutas absolutas	<code>/etc/passwd</code>

3.5 Impacto sobre la tríada CIA

- **Confidencialidad (alto)**: lectura de archivos sensibles: credenciales, código fuente, configuraciones con contraseñas de BD, claves SSH, tokens.
- **Integridad (condicional)**: si la función admite escritura, el mismo defecto puede permitir sobrescribir archivos y llegar a ejecución de código.
- **Disponibilidad (indirecto)**: la información expuesta suele ser el punto de partida de ataques que sí afectan la disponibilidad.

3.6 Clasificación OWASP / CWE

- **CWE-22**: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal').
- **CWE-158**: Improper Neutralization of Null Byte or NUL Character.
- **OWASP Top 10 2021 — A01: Broken Access Control.**

4. Explicación técnica de la vulnerabilidad (caso específico)

La imagen se solicita al endpoint `/image` mediante `filename`:

```
GET /image?filename=68.jpg HTTP/2
Host: <LAB-ID>.web-security-academy.net
```

La aplicación **valida que** `filename` **termine en** `.png`, **pero no rechaza el recorrido** `../../../../` **ni el byte nulo** `%00`. Por eso `../../../../etc/passwd` a secas es rechazado (no termina en `.png`), pero `../../../../etc/passwd%00.png` supera la validación (la cadena termina en `.png`) y, al abrir el archivo, el byte nulo trunca la ruta en `/etc/passwd`, cuyo contenido devuelve el servidor con `Content-Type: image/png`.

5. Procedimiento paso a paso

1. **Iniciar el laboratorio** con «Access the lab» y abrir la tienda desde el navegador de Burp (Proxy → «Open Browser»).
2. **Reconocimiento.** Abrir un producto con «View details» y, en Proxy → HTTP history, localizar la petición de la imagen GET /image
3. **Enviar a Repeater.** Clic derecho sobre la petición /image → «Send to Repeater» (Ctrl+R).
4. **Aplicar el bypass con byte nulo.** Insertar %00 (byte nulo URL-codificado) entre la ruta objetivo y la extensión permitida:

```
GET /image?filename=../../../../etc/passwd%00.png HTTP/2
```

5. **Enviar la petición** con «Send» y revisar el panel Response.
6. **Confirmar la explotación.** La respuesta es HTTP/2 200 OK con Content-Type: image/png y, en el cuerpo, el contenido de /etc/passwd (root:x:0:0:root:/root:/bin/bash, etc.). Queda demostrada la lectura arbitraria de archivos evadiendo la validación de extensión.
7. **Verificar «Lab Solved».** El banner superior cambia de «Not solved» a «Solved».

6. Evidencias — Capturas de pantalla obligatorias

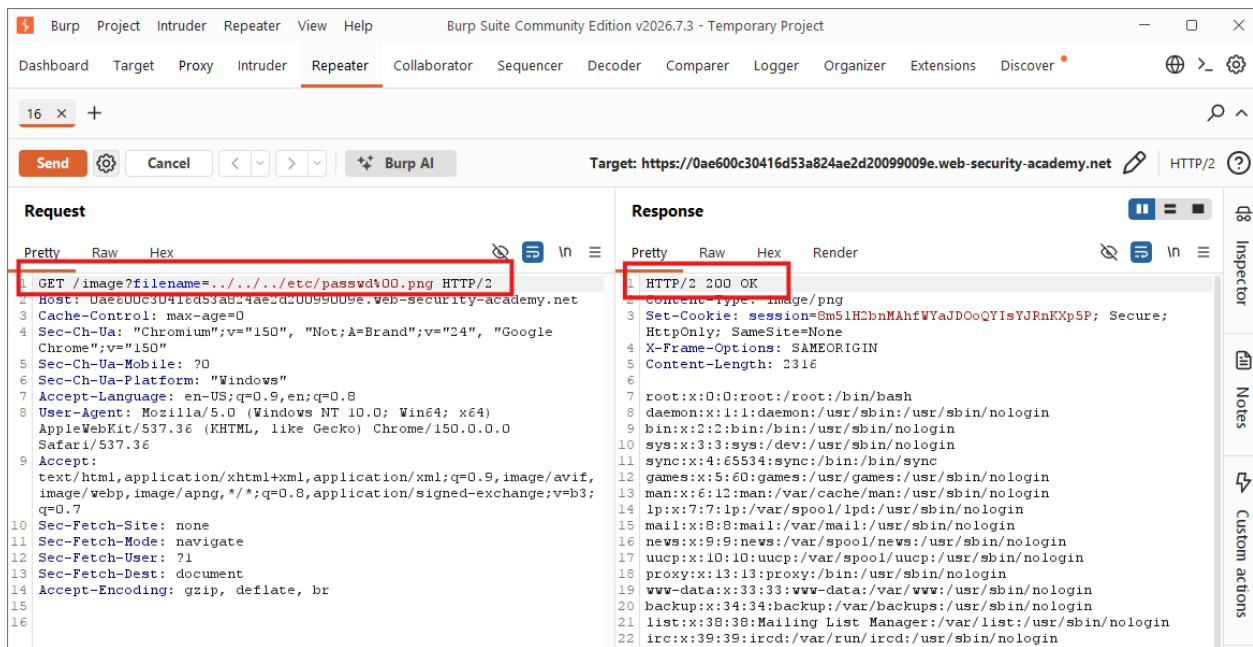
Inserta en cada recuadro tu propia captura tomada durante la ejecución del laboratorio.

6.1 Request interceptado

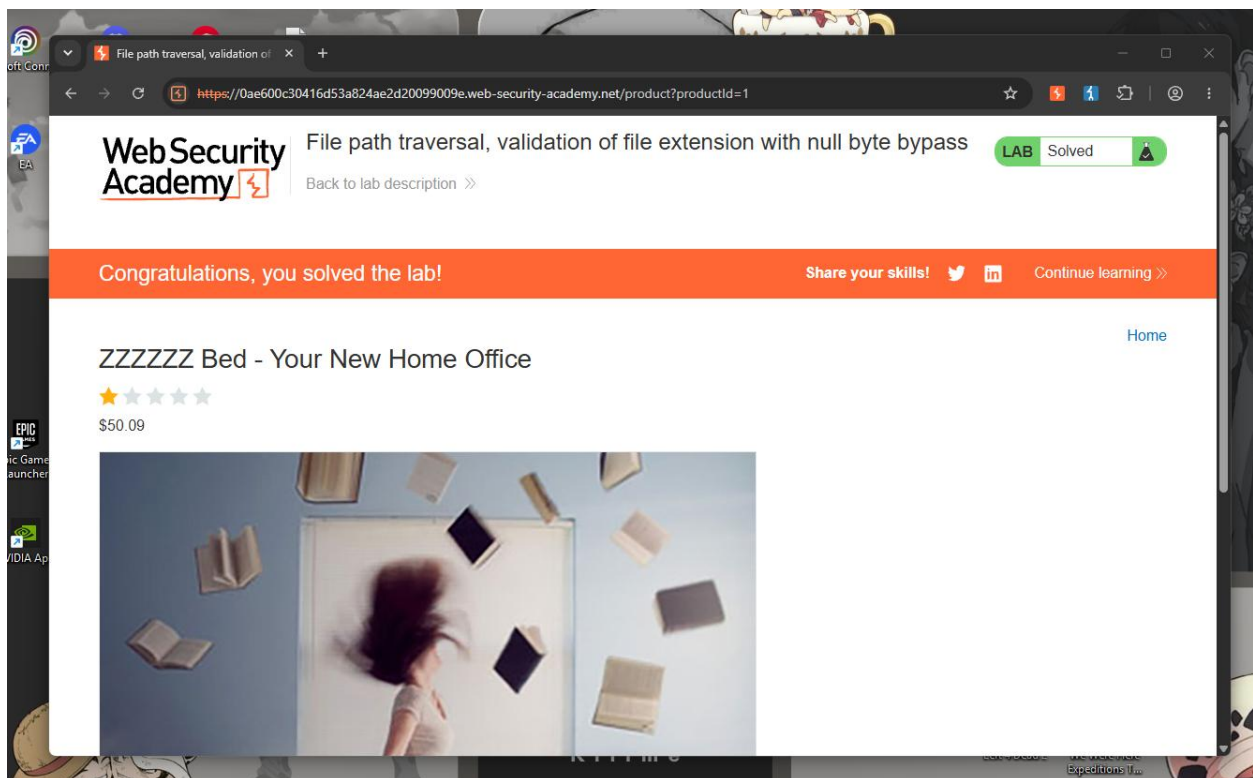
The screenshot shows the Burp Suite Repeater interface. The 'Request' tab is active, displaying a raw HTTP request. The first line of the request is highlighted with a red box: `GET /image HTTP/2`. The rest of the request includes headers like `Host: 0ae600c30416d53a824ae2d20099009e.web-security-academy.net`, `Cache-Control: max-age=0`, `Sec-Ch-Ua: "Chromium";v="150", "Not;A=Brand";v="24", "Google Chrome";v="150"`, `Sec-Ch-Ua-Mobile: ?0`, `Sec-Ch-Ua-Platform: "Windows"`, `Accept-Language: en-US;q=0.9,en;q=0.8`, `User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36`, `Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7`, `Sec-Fetch-Site: none`, `Sec-Fetch-Mode: navigate`, `Sec-Fetch-User: ?1`, `Sec-Fetch-Dest: document`, `Accept-Encoding: gzip, deflate, br`, and `Connection: close`.

The 'Response' tab is also visible, showing an HTTP/2 200 OK response with a `Content-Type: image/png` and a long `Set-Cookie` header. The response body contains the contents of the `/etc/passwd` file, including entries for `root`, `daemon`, `bin`, `man`, `lp`, `mail`, `news`, `uucp`, `proxy`, `www-data`, `backup`, `list`, `irc`, and `gnats`.

6.2 Respuesta vulnerable



6.3 Confirmación «Lab Solved»



7. Explicación del payload

`../../../../etc/passwd%00.png`

El payload engaña a dos capas al mismo tiempo: la validación textual (que exige `.png`) y la apertura real del archivo (que se detiene en el byte nulo):

```
../../../../etc/passwd%00.png
Lo que ve la VALIDACIÓN (cadena completa):
...termina en ".png" -> ACEPTADA
Lo que abre el SISTEMA DE ARCHIVOS (corta en %00):
../../../../etc/passwd -> /etc/passwd
```

Elemento	Función / razón de que funcione
../../../../	Recorrido de directorios hasta la raíz / desde el directorio de imágenes. No está bloqueado en este laboratorio.
etc/passwd	Ruta del archivo objetivo. Legible por cualquier usuario, sirve como prueba de concepto sin exponer secretos críticos.
%00	Byte nulo. Marca «fin de cadena» para la llamada de bajo nivel: la ruta se trunca aquí y se abre /etc/passwd.
.png	Extensión permitida. Su única misión es que la validación textual, que mira el final de la cadena, acepte la petición.

Diferencia con los labs anteriores: aquí el recorrido `../` no está filtrado; la barrera es la extensión, y la clave específica es **el byte nulo**, que aprovecha el desajuste entre la validación de alto nivel y la lectura de archivos de bajo nivel.

8. Mitigación recomendada

La validación de extensión es, por sí sola, una defensa débil. Recomendaciones:

- **Rechazar bytes nulos:** descartar cualquier entrada que contenga `%00` / `\0` antes de procesarla.
- **Usar plataformas actualizadas:** las versiones modernas de los lenguajes y librerías ya no truncan las rutas por byte nulo; mantener el runtime al día elimina el vector.
- **Canonicalizar y verificar el contenedor:** resolver la ruta final (`realpath` / `getCanonicalPath`) y comprobar que el archivo resuelto sigue dentro del directorio base permitido. **Este control detiene el recorrido aunque la extensión parezca válida.**
- **Validación por lista blanca:** aceptar solo nombres que cumplan un patrón estricto de nombre COMPLETO, p. ej. `^[a-zA-Z0-9_-]+\.(png|jpg)$`, que no admite `../` ni `%00`.
- **Referencias indirectas:** exponer un identificador (`imageId=68`) mapeado internamente a la ruta física, evitando que el cliente controle nombre y extensión.
- **Mínimo privilegio:** ejecutar el servicio con un usuario sin acceso de lectura a archivos sensibles del sistema, para limitar el impacto si el control falla.

9. Conclusión

El laboratorio demuestra que validar la **extensión de un archivo es insuficiente** cuando la comprobación es textual y la lectura de archivos es susceptible a truncamiento por byte nulo. Con el payload `../../../../etc/passwd%00.png` se superó la validación (la cadena termina en `.png`) mientras el `%00` truncaba la ruta real en `/etc/passwd`, logrando su lectura con respuesta `200 OK`. Cierra así la serie de path traversal de nivel Practitioner con una conclusión común: las defensas basadas en filtrar o validar patrones concretos son evadibles; la protección robusta es **canonicalizar la ruta y verificar que el archivo**

resuelto permanezca dentro del directorio permitido, además de rechazar bytes nulos y mantener las plataformas actualizadas.